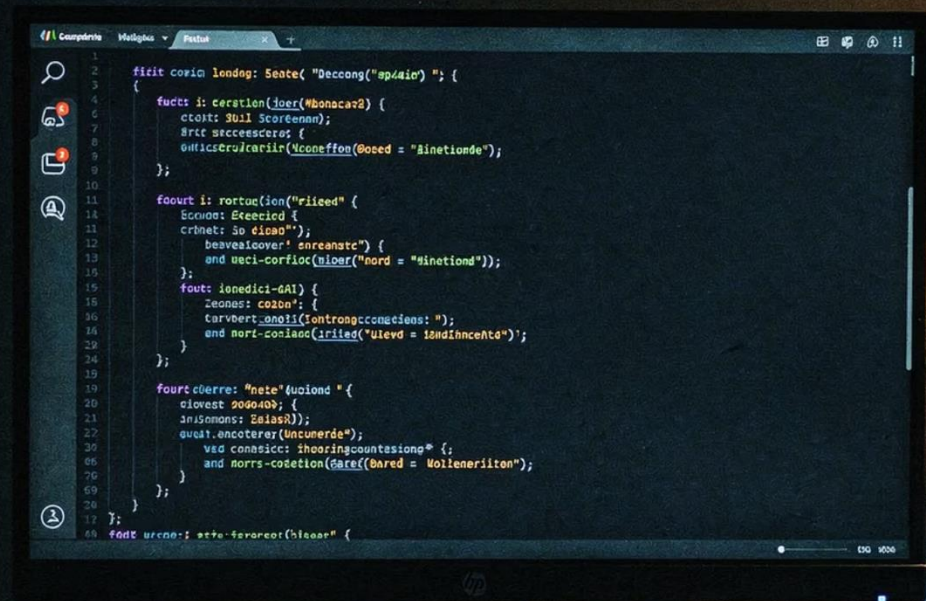


Angular Moderno

12 - Introdução ao Router



Angular Moderno

12 - Introdução ao Router

Contexto do Projeto do Curso

Nosso projeto é baseado na construção de um **e-commerce real**, evoluindo aula a aula. Com o que foi feito até a aula passada, o projeto já possui uma base sólida e funcional.

Catálogo de Produtos

Listagem de produtos renderizada com componentes standalone e Signals.

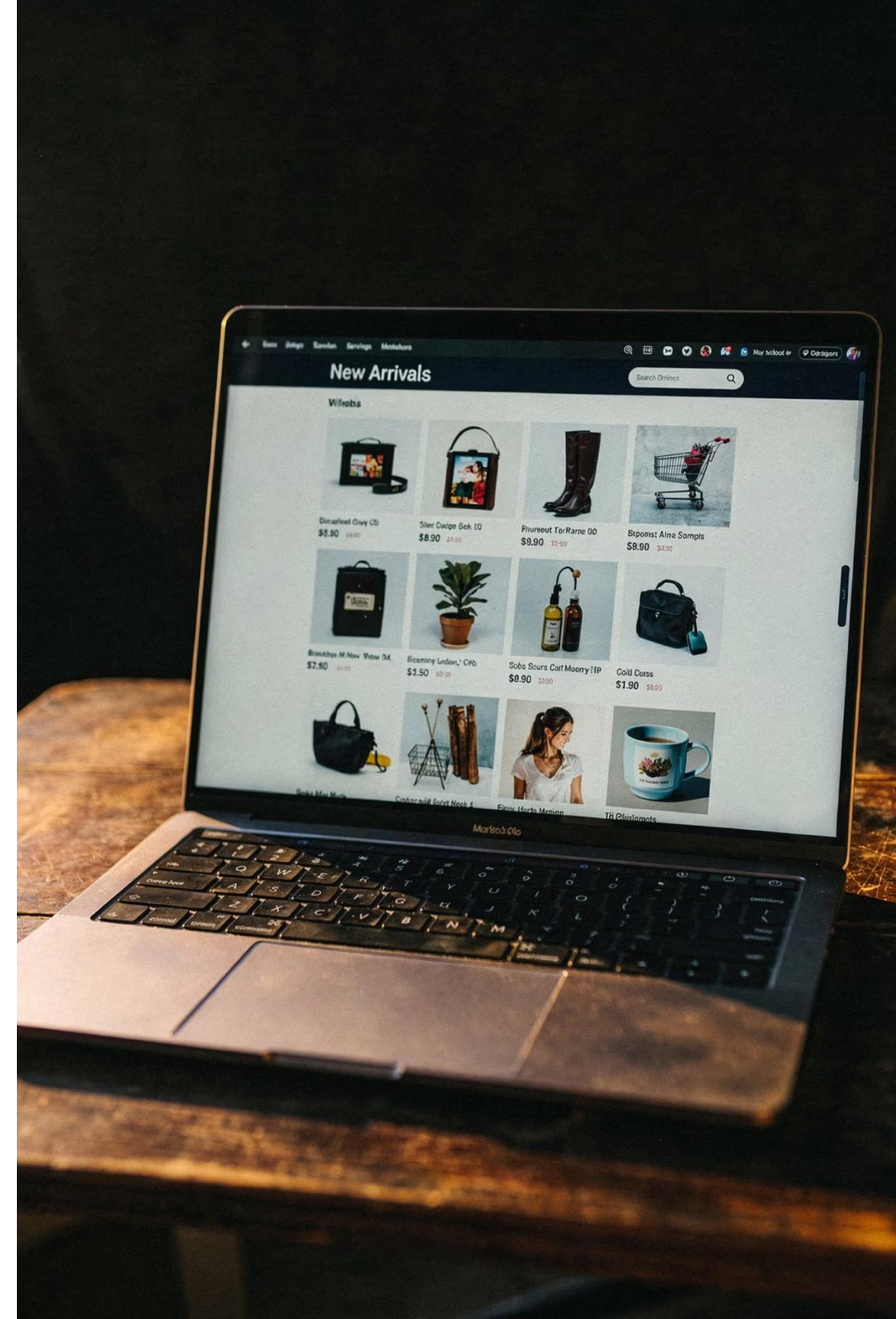
Carrinho Funcional

Carrinho implementado com Signals **intencionalmente dentro do componente** como etapa didática intermediária.

Uma Única Tela

Todo o conteúdo acontece em um único componente.
Funciona, mas não escala.

- ❗ O carrinho no componente é intencional. Será refatorado para um Service em aulas futuras. Por enquanto, foco é o Router.



Objetivo da Aula

Ao final desta aula, você será capaz de transformar sua aplicação Angular em uma **SPA (Single Page Application) com múltiplas páginas**, usando o Router moderno — sem NgModule, sem legado.



Configurar rotas

Definir rotas no Angular moderno usando `provideRouter` e `app.routes.ts`.



Entender o Router

Compreender o papel do Router: qual componente aparece em cada URL.



Navegar com routerLink

Criar menus e links de navegação sem recarregar a página.



Usar router-outlet

Definir onde o Angular renderiza o componente da rota ativa.

Onde Estamos no Projeto

Antes de avançar, é importante entender o ponto de partida. Sua aplicação atual já é funcional — mas tudo acontece em um único lugar.

Signals funcionando

Estado reativo com `signal()` e `computed()` aplicados ao catálogo e ao carrinho.

Carrinho implementado

Adicionar, remover e calcular total — tudo dentro do componente principal.

Um único componente

O `App` controla toda a lógica e toda a exibição. Funciona para demos, não para produção.

⊗ Toda a aplicação ainda depende de um único componente (`App`)

Product List

Item 01

Item 02

Item 03

Item 04

O Problema Atual

Sua aplicação não tem **navegação real**. Tudo acontece na mesma tela, independente do que o usuário faz. Isso cria problemas sérios em aplicações reais:

- Não é possível compartilhar a URL de uma página específica
- O botão "voltar" do navegador não funciona como esperado
- Toda a interface fica visível ao mesmo tempo
- A aplicação não escala: adicionar novas seções vira um caos

URL: localhost:4200

Produtos visíveis

URL: localhost:4200

Carrinho também visível

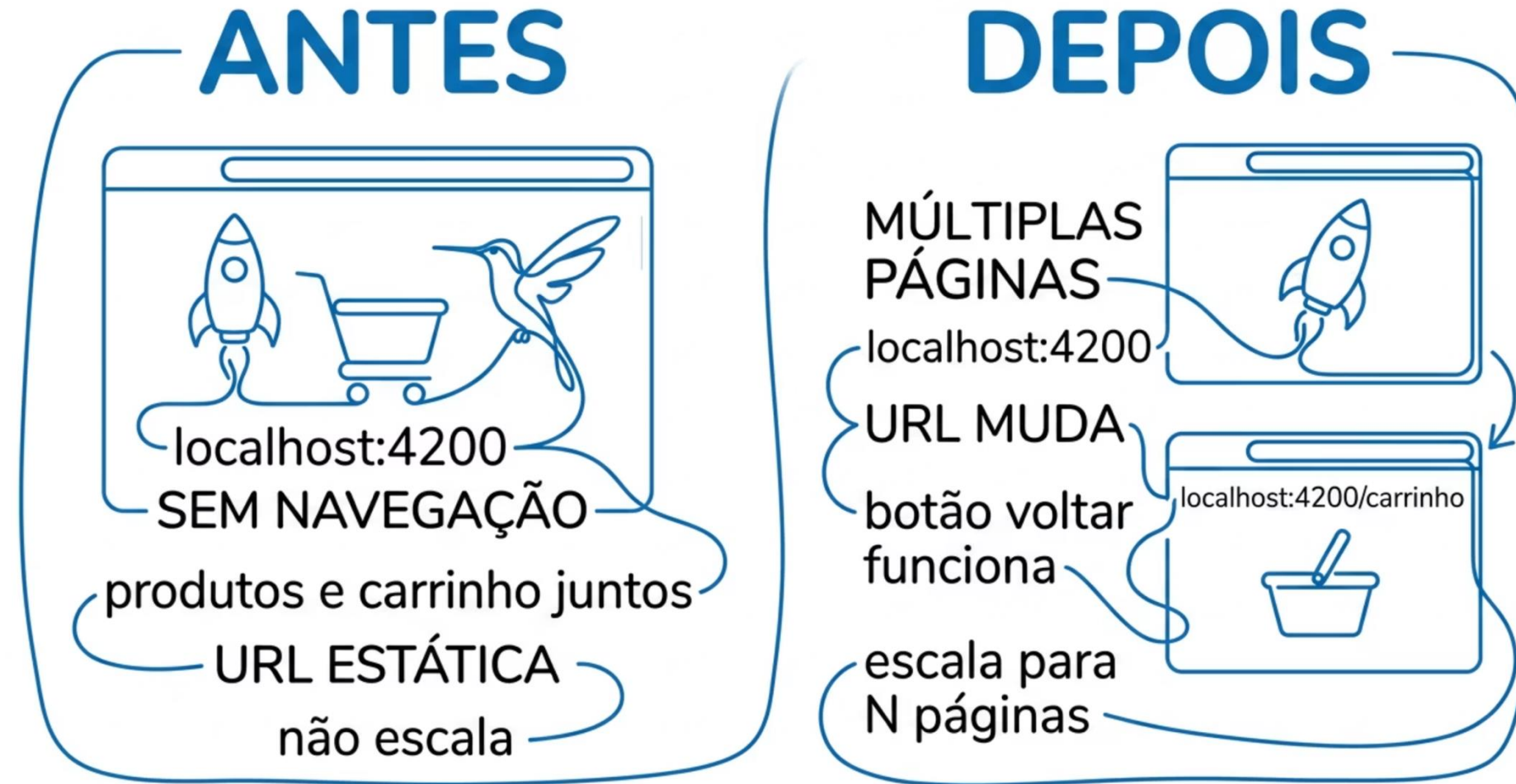
URL: localhost:4200

A URL nunca muda

❌ Uma aplicação real precisa que cada "página" tenha sua própria URL. Sem isso, a experiência do usuário é quebrada.

O Que Vamos Resolver

Vamos transformar a aplicação de um monobloco em uma SPA com páginas independentes e URLs próprias.



Cada rota renderiza um componente diferente. O restante da aplicação permanece intacto — apenas o `router-outlet` troca o que está visível.

O Que é o Router?

"O Router decide qual componente aparece com base na URL."

Pense no Router como um **controlador de tráfego**: ele lê a URL do navegador e mapeia essa informação para o componente correto. Você define as regras (as rotas), e o Angular executa automaticamente.



Lê a URL

O Router observa cada mudança de URL, incluindo navegação com histórico do navegador.



Encontra a rota

Compara a URL com a tabela de rotas que você configurou em `app.routes.ts`.



Renderiza o componente

Injeta o componente correspondente no `router-outlet` do template.



Importante: Mudança de Mentalidade

ANTES (Angular Clássico) — App controla a tela

O App decide o que mostrar usando `*ngIf`, flags booleanas ou lógica no TypeScript. A URL não tem significado.

```
// App controlando tudo
mostrarCarrinho = false;

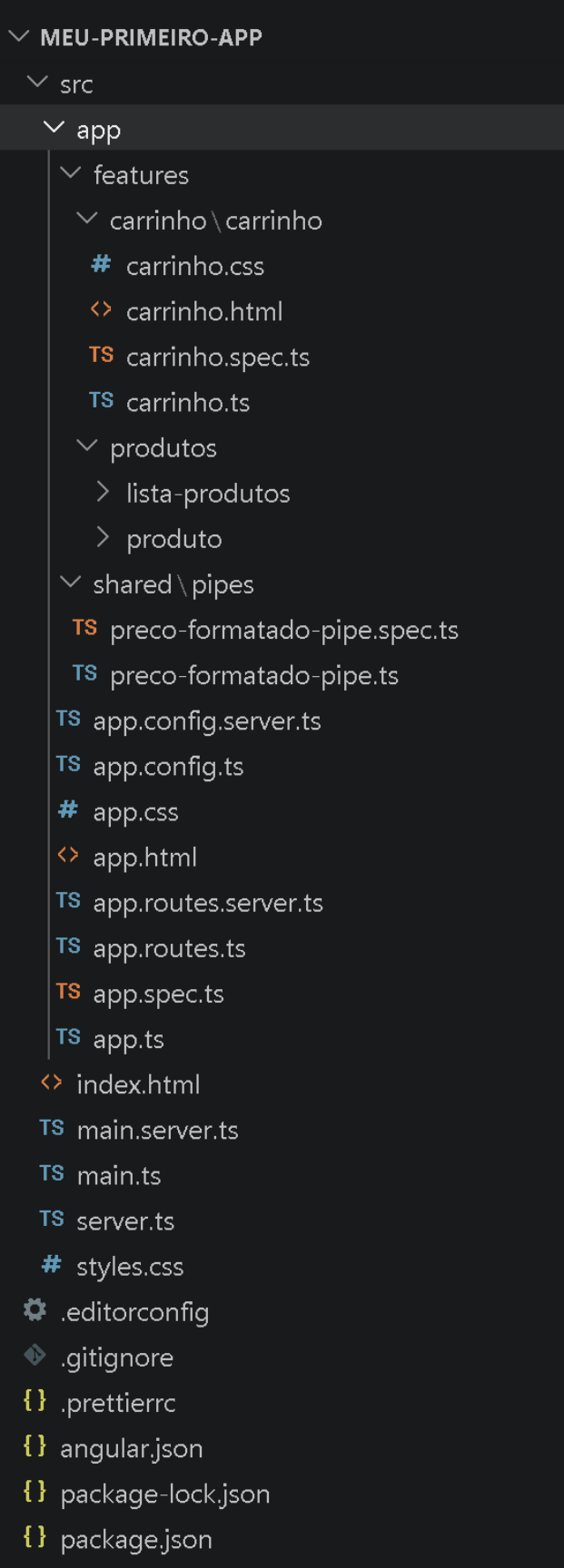
toggleCarrinho() {
  this.mostrarCarrinho =
    !this.mostrarCarrinho;
}
```

AGORA (Angular Moderno) — URL controla a tela

A URL é a fonte da verdade. O Angular lê a URL e renderiza o componente correto, sem lógica condicional no App.

```
// app.routes.ts decide tudo
export const routes: Routes = [
  { path: '', component: ListaProdutos },
  { path: 'carrinho', component: Carrinho }
];
```

📌 "O App não controla mais a tela. A URL controla." Esta é a frase mais importante desta aula. Grave isso.



Nova Estrutura do Projeto

Com o Router, passamos a organizar a aplicação por **responsabilidade de navegação**.

Cada feature tem sua própria pasta dentro de `features/`.

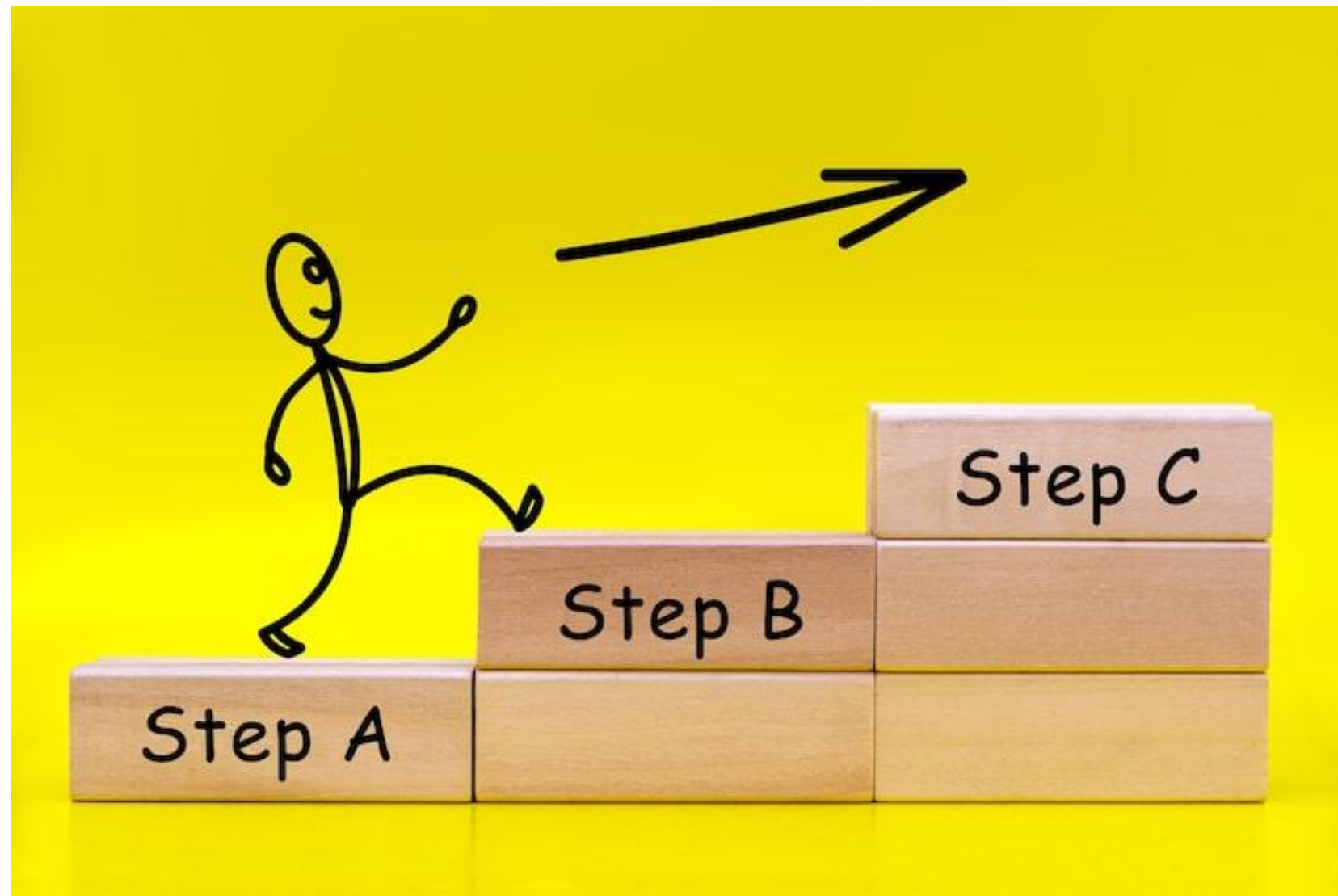
Por que organizar assim?

- Cada feature é **autocontida**: seus arquivos ficam juntos
- Facilita Lazy Loading no futuro (cada feature vira um chunk separado)
- Escalabilidade: adicionar uma nova página é adicionar uma nova pasta
- Time pode trabalhar em features independentes sem conflitos

 Essa estrutura é o padrão recomendado para projetos Angular modernos em produção.

Visão Geral da Parte Prática

Agora vamos colocar a mão no código. São **9 passos**. Siga na ordem exata para evitar erros de dependência.



PASSO 1

Criar o Componente Carrinho

Use o Angular CLI para gerar o componente dentro da pasta correta. O CLI cria todos os arquivos necessários automaticamente.

```
ng generate component features/carrinho/carrinho
```

Ou usando o atalho:

```
ng g c features/carrinho/carrinho
```

Arquivos gerados pelo CLI

```
features/carrinho/  
├── carrinho.ts  
├── carrinho.html  
├── carrinho.css  
└── carrinho.spec.ts
```

O que o CLI faz

- Cria os 4 arquivos do componente
- Gera o selector baseado no nome
- Não modifica NgModule (não existe!)
- Os componentes já são standalone por padrão nas versões modernas do Angular

Ajustar carrinho.ts

Configure o componente gerado. Por enquanto, ele será simples — o carrinho completo vem em aulas futuras.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-carrinho',
  imports: [],
  templateUrl: './carrinho.html',
  styleUrls: ['./carrinho.css']
})
export class Carrinho {
  // Lógica do carrinho virá em aulas futuras
  // Por ora, apenas exibe a página de carrinho
}
```

 Os componentes já são standalone por padrão nas versões modernas do Angular.

PASSO 3

Ajustar carrinho.html

O template inicial deve ser simples e funcional, apenas para confirmar que a rota está funcionando. Não adicione lógica complexa agora.

```
<h1>Carrinho</h1>
```

```
<p>Você está na página do carrinho.</p>
```

```
<p>Nas próximas aulas, este carrinho será conectado ao estado global.</p>
```

Configurar Rotas — app.routes.ts

❏ **Importante:** Se o arquivo `app.routes.ts` não existir no projeto, crie-o. Se já existir (criado pelo CLI ao inicializar o projeto), apenas edite o conteúdo.

```
import { Routes } from '@angular/router';
import { ListaProdutos } from '../features/produtos/lista-produtos/lista-produtos';
import { Carrinho } from '../features/carrinho/carrinho/carrinho';

export const routes: Routes = [
  {
    path: '',
    component: ListaProdutos,
  },
  {
    path: 'carrinho',
    component: Carrinho,
  },
];
```

⊗ **Erro comum:** Esquecer de importar os componentes no topo do arquivo. O TypeScript vai reclamar! Preste atenção nos caminhos.

⊗ **Importante:** As rotas são avaliadas de cima para baixo!

Entendendo a Tabela de Rotas

Cada objeto dentro do array `routes` é uma rota. O Router percorre esse array de cima para baixo até encontrar um `path` que corresponde à URL atual.

path	Componente renderizado	URL no navegador
<code>' '</code> (vazio)	<code>ListaProdutos</code>	<code>localhost:4200/</code>
<code>'carrinho'</code>	<code>Carrinho</code>	<code>localhost:4200/carrinho</code>

Por que path vazio (")?

O `path: ' '` representa a rota raiz, que é quando não há nada após o domínio. É a "página inicial" da sua aplicação.

E rotas não encontradas?

Você pode adicionar

```
{ path: '**', redirectTo: '' }
```

para redirecionar URLs inválidas.

Faremos isso em aulas futuras.

Verificar app.config.ts

O `app.config.ts` é onde você registra os provedores globais da aplicação. O Router precisa estar aqui via `provideRouter`.

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';
import { provideClientHydration, withEventReplay } from '@angular/platform-browser';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideClientHydration(withEventReplay())
  ]
};
```

✔ **Boa notícia:** Se você criou o projeto com `ng new` e escolheu adicionar routing, este arquivo já existe com `provideRouter(routes)` configurado. Apenas verifique se está igual ao código acima.

❗ **Por que `provideRouter` e não `RouterModule`?** O `provideRouter` é a API funcional moderna do Angular. Ele substitui o antigo `RouterModule.forRoot()` e funciona sem `NgModule`.

❗ Seu projeto pode conter outros providers! Não remova, apenas garanta que **`provideRouter(routes)`** esteja presente.

Ajustar app.ts — Faça Isso Antes do HTML!

⊗ **Ordem importa:** Ajuste o TypeScript ANTES de editar o template HTML. Se você adicionar `routerLink` no HTML sem importar primeiro, o Angular vai lançar erros difíceis de diagnosticar.

```
import { Component, signal } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('meu-primeiro-app');
}
```

Os dois imports essenciais: `RouterOutlet` (onde o componente aparece) e `RouterLink` (diretiva para links de navegação).

Remover ListaProdutos dos imports

Este passo é importante e frequentemente causa confusão. Você precisa **remover** o `ListaProdutos` dos imports do `App`.

ANTES (errado a partir de agora)

```
imports: [  
  ListaProdutos  
]
```

DEPOIS (correto)

```
imports: [  
  RouterOutlet,  
  RouterLink  
]
```

Por que remover?

O `App` não vai mais usar o `ListaProdutos` diretamente no seu template — o Router vai instanciá-lo automaticamente quando a rota `/` for acessada. Se você mantiver no `imports` sem usar no template, o Angular vai alertar sobre imports desnecessários.

⚠ Não esqueça também de remover o import do TypeScript no topo do arquivo.

Deixar imports mortos gera warnings de compilação.

Ajustar app.html

O template do `App` agora é apenas o **esqueleto da aplicação**: um menu fixo e o espaço dinâmico onde as páginas aparecem.

```
<h1>Minha Loja</h1>

<nav>
  <a routerLink="/">Produtos</a> |
  <a routerLink="/carrinho">Carrinho</a>
</nav>

<hr />

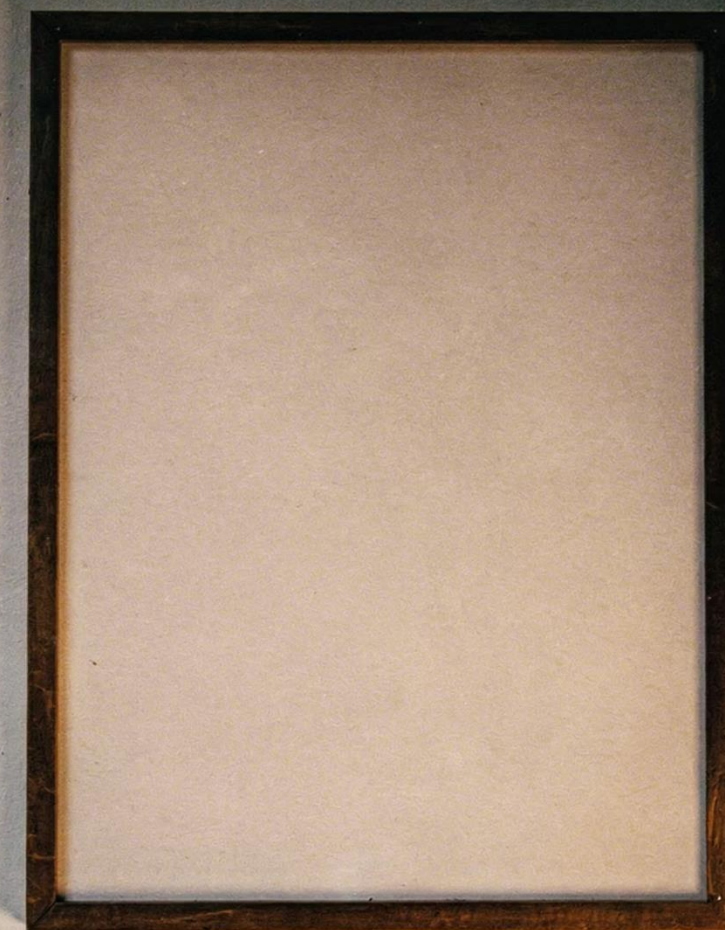
<router-outlet></router-outlet>
```

O router-outlet em Detalhe

"O router-outlet é o espaço reservado onde o Angular injeta o componente da rota ativa."

Pense no `<router-outlet>` como um **quadro vazio na parede**. O Router é o curador que decide qual quadro (componente) pendurar ali, baseado na URL. Quando você navega, o quadro é trocado — sem recarregar a página.

❗ O `<router-outlet>` não renderiza nada por si só. Ele apenas marca o local onde o componente da rota será inserido no DOM.



PASSO 9

Rodar o Projeto

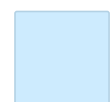
Com todos os passos concluídos, é hora de iniciar o servidor de desenvolvimento e verificar se tudo funciona.

```
ng serve
```

Aguarde a compilação.

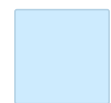
Testar a Aplicação

Abra o navegador em `localhost:4200` e faça este checklist completo. Cada item confirma que uma parte do Router está funcionando corretamente.



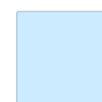
Página inicial carrega

Acesse `localhost:4200/`. O catálogo de produtos deve aparecer normalmente, sem erros no console.



URL muda no navegador

Observe a barra de endereços: ela deve mudar de `/` para `/carrinho` ao navegar.



Link "Carrinho" funciona

Clique em "Carrinho" no menu. A página do carrinho deve aparecer sem recarregar a página.



Botão "Voltar" funciona

Pressione o botão voltar do navegador. Você deve retornar à página anterior. O histórico de navegação está funcionando!

O Que Acabou de Acontecer

Parabéns! Sua aplicação agora é uma SPA profissional com navegação real. Veja o que mudou estruturalmente:

Navegação Real

Cada página tem uma URL própria. O usuário pode compartilhar links, usar favoritos e o histórico do navegador funciona corretamente.

Separação de Responsabilidades

O **App** só cuida do layout global. Cada feature cuida de si mesma. O Router faz a orquestração.

Base para SPA Profissional

Esta estrutura suporta Lazy Loading, Guards, Resolvers e todas as features avançadas que veremos nas próximas aulas.

📌 Neste momento, o carrinho ainda NÃO é global

BEFORE

AFTER



Conexão com a Aula Anterior

Você pode estar se perguntando: "E o carrinho que construímos na aula passada?"

Boa pergunta. Ele ainda está lá, e isso é **intencional**.

Estado atual (intencional)

- Carrinho ainda no componente
- Estado local com Signals
- Não é um Service ainda
- Não compartilhado globalmente

Por que não refatorar agora?

A aula de hoje foca em uma única habilidade: **o Router**. Misturar a refatoração do carrinho criaria complexidade desnecessária e dificultaria o aprendizado.

O carrinho será movido para um **Service com estado global** em uma aula dedicada a isso.

- ❏ Didática progressiva: cada aula resolve um problema de cada vez. Não avance por conta própria! você pode criar problemas que dificultarão o aprendizado futuro.

Ajustes Finais e Organização do Código

Antes de encerrar a aula, faça uma revisão rápida do código. Código limpo é tão importante quanto código funcional.

- Remover imports não usados
Verifique o topo de cada arquivo `.ts`. Se você importou `ListaProdutos` no `app.ts` e removeu do array `imports`, remova também o `import` no topo.
- Verificar paths de importação
Confirme que os caminhos em `app.routes.ts` batem com a localização real dos arquivos. Um caractere errado gera erro de compilação.
- Rodar `ng serve` novamente
Após os ajustes, reinicie o servidor para garantir que tudo compila limpo, sem warnings ou erros no terminal.

❗ Use o VSCode com a extensão **Angular Language Service** para identificar imports desnecessários automaticamente com sublinhado amarelo.

Checklist Final da Aula

Antes de fechar o projeto, confirme que cada item abaixo está funcionando. Se algum falhar, volte ao passo correspondente.

Configuração

- ✓ Componente `Carrinho` criado
- ✓ `app.routes.ts` configurado com 2 rotas
- ✓ `provideRouter(routes)` em `app.config.ts`
- ✓ `RouterOutlet` e `RouterLink` importados no App
- ✓ `ListaProdutos` removido dos imports do App

Funcionamento

- ✓ Template com menu, `routerLink` e `router-outlet`
- ✓ Rota `/` renderiza `ListaProdutos`
- ✓ Rota `/carrinho` renderiza `Carrinho`
- ✓ URL muda ao navegar
- ✓ Botão voltar do navegador funciona
- ✓ Nenhum erro no console do navegador

Erros Comuns e Como Resolver

Estes são os erros mais frequentes nesta aula. Se algo der errado, consulte esta lista antes de pedir ajuda.

✗ "Can't bind to routerLink since it isn't a known property"

Causa: RouterLink não está no array `imports` do componente que usa o atributo no template.

Solução: Adicione RouterLink ao array `imports` do componente afetado.

✗ Página em branco ao navegar

Causa: `<router-outlet>`
`</router-outlet>` não está no template, ou RouterOutlet não está nos imports.

Solução: Verifique o `app.html` e o array `imports`.

✗ "Cannot find module './features/...'"

Causa: O caminho de importação em `app.routes.ts` está errado — letra maiúscula, pasta errada ou nome diferente.

Solução: Copie o caminho exato do Explorer do VSCode.

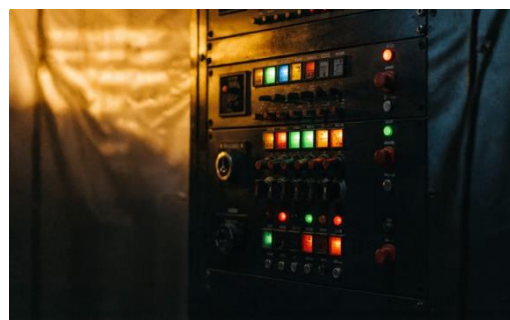
Conceitos-Chave para Fixar

Estes são os três pilares do que você aprendeu hoje. Se você entender estes, o restante do Router fará sentido naturalmente.



Rotas =
Mapeamento URL → Componente

Cada rota é uma regra: "quando a URL for X, mostre o componente Y". Você define as regras em `app.routes.ts`.



`provideRouter` = Registro do Router

O `provideRouter(routes)` em `app.config.ts` ativa o Router na aplicação e injeta as rotas no sistema.

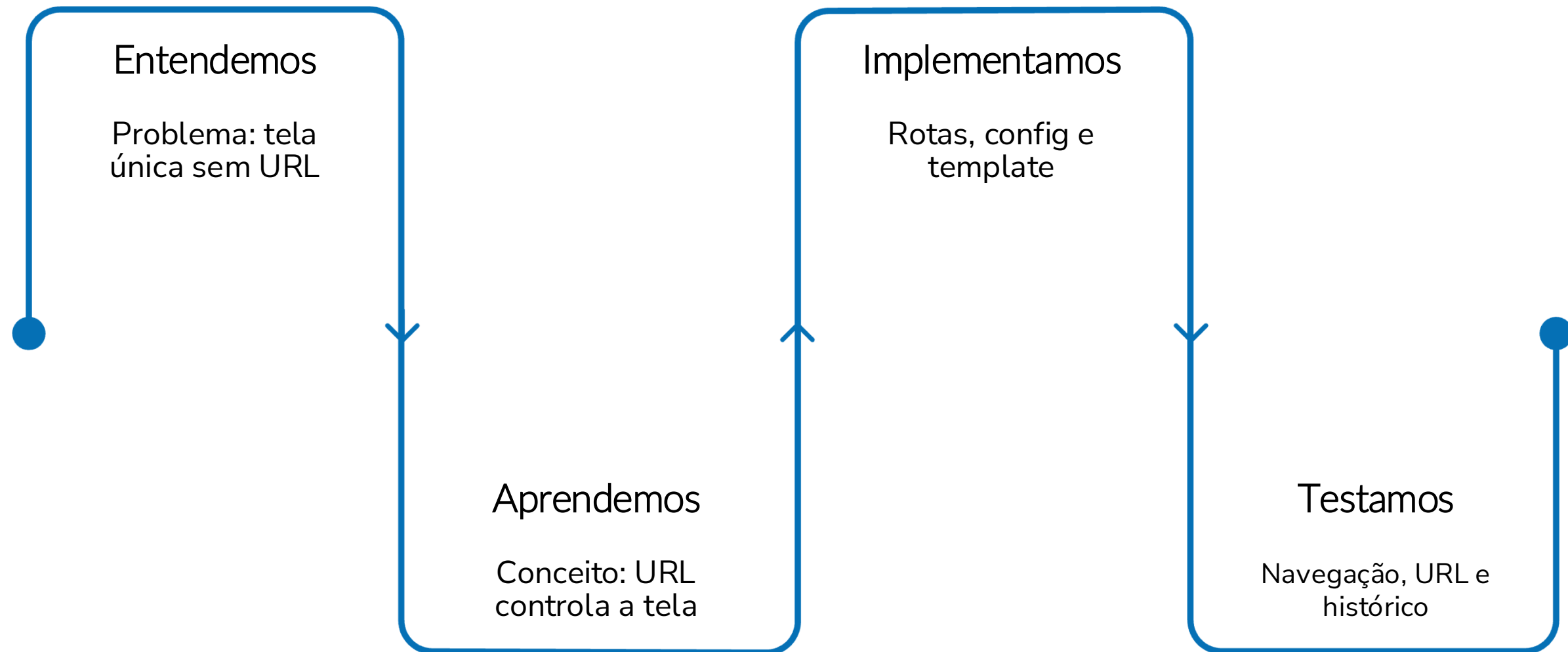


`router-outlet` = Slot Dinâmico

O `<router-outlet>` é o espaço onde o Angular injeta o componente ativo. É trocado a cada mudança de rota.

Recapitulando: a Jornada de Hoje

Você saiu de uma aplicação de tela única para uma SPA profissional com roteamento real. Veja todo o caminho percorrido:



A transformação mais importante não foi no código. Foi na **mentalidade**! Você agora pensa em termos de rotas e URLs, não de flags e condicionais. Isso é pensar como um desenvolvedor Angular profissional.

Encerramento

"O Angular Router não é apenas uma ferramenta de navegação. É a espinha dorsal de qualquer aplicação de produção. Hoje você instalou essa espinha."

O que você fez

Transformou uma aplicação monolítica em uma SPA com múltiplas páginas, URLs reais e navegação profissional.

A frase da aula

"O App não controla mais a tela. A URL controla." Essa é a mudança de paradigma do Router moderno.

Dúvidas

?

